Python Practice Set: Lists, Tuples & Sets

Real-world tasks using everything you have learned so far.

Section A — List

A **list** is an ordered, changeable collection that allows duplicates. Think of it as a queue or a shelf you can freely add to, remove from, or rearrange.

Key methods you will use: **append()** adds one item to the end, **remove()** deletes the first match, **len()** counts items, and you access any item by its position (index starts at 0, e.g. cart[0]).

Task A1 — Supermarket Self-Checkout Cart

Imagine you are at a self-checkout machine. You scan items one by one, occasionally remove something you changed your mind about, and finally get a bill. Build exactly that.

- Start with an empty list called **cart**.
- Use a **while True** loop to keep asking the user: "Enter item name (or 'done' to checkout): ".
- If the user types **done**, break out of the loop.
- If the user types **remove**, ask which item to remove. If that item is in the cart, remove it and print "Removed from cart". If not found, print "Item not in cart."
- Otherwise, append the entered item to the cart and print "Added. Items in cart: X" where X is the current length of the cart.
- After the loop, use a **for loop** to print every item in the cart with its number, like: "1. rice", "2. milk" etc.
- At the end print: "Total items purchased: X"

Sample run:

```
Enter item (or 'done'): rice
Added. Items in cart: 1
Enter item (or 'done'): milk
Added. Items in cart: 2
Enter item (or 'done'): remove
Which item to remove? milk
Removed from cart.
Enter item (or 'done'): bread
Added. Items in cart: 2
Enter item (or 'done'): done

Your cart:
1. rice
2. bread
Total items purchased: 2
```

Hint: use if item in cart to check membership before removing.

Task A2 — Student Marks Manager

A teacher wants to enter marks for 5 students, then see the highest, lowest, and average — all from a list.

- Create an empty list called **marks**.
- Use a **for loop** running 5 times. Each iteration ask: "Enter marks for Student X: " and append the integer to marks.
- After the loop: print the highest mark using **max(marks)**, the lowest using **min(marks)**, and the average (sum(marks) / len(marks)) rounded to 2 places.
- Loop through the marks list again. If a student's mark is below 40, print "Student X — FAIL", otherwise "Student X — PASS".

Sample output after entering 85, 40, 33, 90, 76:

```
Highest: 90   Lowest: 33   Average: 64.8

Student 1 - PASS
Student 2 - PASS
Student 3 - FAIL
Student 4 - PASS
Student 5 - PASS
```

Section B — Tuple

A **tuple** is an ordered, *unchangeable* collection. Once you create it, you cannot add, remove, or edit its items. Use tuples for data that must stay fixed — like coordinates, product details, or official records.

Key facts: created with round brackets, e.g. **item = ("rice", 50, "Staples")**. You read values by index: item[0] is "rice", item[1] is 50. You can loop over a tuple the same way as a list.

Task B1 — Product Catalogue Display

A store has a fixed price catalogue — prices never change mid-session. Store it as a tuple of tuples and let the user look up a product.

- Create a variable **catalogue** that is a tuple containing 5 inner tuples. Each inner tuple holds three values: (product_name, price, category). Example: ("rice", 50, "Staples"), ("milk", 60, "Dairy"), etc. Fill in five products of your choice.
- Print the full catalogue using a **for loop**: "Name: rice | Price: 50 | Category: Staples"
- Then ask the user: "Search product: " and use another for loop to check if any product in the catalogue matches (case-insensitive). If found, print its full details. If not found, print "Product not in catalogue."

Sample output:

```
---- Catalogue ----
Name: rice     | Price: 50  | Category: Staples
Name: milk     | Price: 60  | Category: Dairy
...

Search product: Milk
```

```
Found! milk - Rs.60 - Dairy
```

Hint: use product[0].lower() == search_term.lower() for case-insensitive matching.

Task B2 — Bus Seat Booking (read-only records)

A mini bus has seats already assigned and stored as tuples. Your program checks whether a seat is taken.

- Create a tuple called **booked_seats** with 6 seat numbers already booked, e.g. (3, 7, 12, 15, 20, 24).
- Ask the user to enter a seat number they want to book (convert to int).
- Use an **if** condition with the **in** keyword to check: if the seat is in booked_seats, print "Seat X is already booked.", otherwise print "Seat X is available. Booking confirmed!"
- Wrap this in a **while True** loop and keep asking until the user types **0** to exit.
- Also print how many seats are still free: total seats = 30, booked = len(booked_seats).

Section C — Set

A **set** automatically removes duplicates and does not care about order. It is perfect when you only need to know *whether* something exists, not how many times.

Key methods: **add()** adds an item (ignored if already present), **discard()** removes an item safely, **len()** counts unique items. Two powerful set operations: **&** finds items in both sets (intersection), **|** combines both sets without duplicates (union).

Task C1 — Event Unique Visitor Counter

A college festival scans visitor badges at the gate. The same person might try to scan twice. Your job is to count only unique visitors.

- Create an empty **set** called **visitors**.
- Use a **while True** loop asking: "Scan visitor name (or 'done' to end): ".
- If done, break.
- Otherwise, add the name (lowercased) to the set. If the name was already in the set before adding, print "Already scanned — duplicate ignored." (Hint: check membership with **in** before adding.)
- If it is a new visitor, print "Welcome, [name]!"
- After the loop, print the total unique visitor count using **len(visitors)**, and print all visitor names using a for loop.

Sample run:

```
Scan visitor name: aman
Welcome, aman!
Scan visitor name: raj
Welcome, raj!
Scan visitor name: aman
```

```
Already scanned - duplicate ignored.
Scan visitor name: done

Total unique visitors: 2
aman
raj
```

Task C2 — Common Skills Finder (Set Intersection)

A recruiter has two candidates and wants to instantly see which skills they share and which skills either of them has.

- Ask for Candidate 1's name, then ask how many skills they have (an integer n). Use a for loop running n times to collect each skill and add it to a set called **skills1**. Lowercase each skill before adding.
- Repeat the same process for Candidate 2, storing into **skills2**.
- Print **Common skills** using the & operator: skills1 & skills2
- Print **All skills combined** using the | operator: skills1 | skills2
- Print **Skills only Candidate 1 has**: skills1 - skills2
- Print **Skills only Candidate 2 has**: skills2 - skills1

Sample output:

```
Candidate 1 skills: {'python', 'sql', 'excel'}
Candidate 2 skills: {'python', 'java', 'excel'}

Common skills      : {'python', 'excel'}
All combined       : {'python', 'sql', 'excel', 'java'}
Only Candidate 1   : {'sql'}
Only Candidate 2   : {'java'}
```

Final Task — Mini Hospital Reception System

This task uses lists, tuples, and sets together in one real-world scenario. Read the full description carefully before writing any code.

Scenario:

A small hospital reception needs a program to manage patient check-ins. Doctor details are fixed (tuple). The waiting queue changes constantly (list). A visited log tracks who already came today to avoid double-entry (set).

What to build:

- **Tuple — Doctor Info (fixed):** Create a tuple called **doctor** with three values: (doctor_name, specialisation, room_number). Example: ("Dr. Mehta", "Cardiology", 204). Print the doctor's details at the start: "Welcome to [name]'s clinic — [spec], Room [no]"

- **Set — Visited log:** Create an empty set called **visited**. Every time a new patient checks in, add their name (lowercased) to this set. If the name already appears in the set, print "[name] has already visited today. Skipping." and do NOT add them to the queue again.
- **List — Waiting queue:** Create an empty list called **queue**. Add new (non-duplicate) patients to this list using append().
- **Main loop (while True):** Show a menu each time: "1. Check in patient 2. Call next patient 3. View queue 4. Exit". Ask the user to pick an option.
- **Option 1 — Check in:** Ask for the patient's name. Check the visited set. If new, add to both set and queue and print "[name] added to queue. Position: X" (X = their position, i.e. len(queue)).
- **Option 2 — Call next:** If the queue is not empty, remove the first patient using queue.pop(0) and print "Calling [name] — please proceed to Room [room_number]". If queue is empty, print "No patients waiting."
- **Option 3 — View queue:** If empty, print "Queue is empty". Otherwise loop through the list and print each patient with their position number.
- **Option 4 — Exit:** Print "Session ended. Total unique patients today: X" using len(visited), then break.

Hint: queue.pop(0) removes and returns the first item — that is how a real queue works (first in, first out).

Checklist — before you call it done

- Supermarket cart correctly handles both 'remove' and normal items without crashing.
- Marks manager shows correct highest, lowest, average, and PASS/FAIL for each student.
- Catalogue search works regardless of whether the user types uppercase or lowercase.
- Bus booking correctly uses the 'in' keyword on a tuple.
- Visitor counter never counts the same name twice.
- Skills finder correctly prints intersection, union, and differences.
- Hospital system: duplicate patient is caught by the set, not added to the list queue again.
- Hospital option 2 removes from the front of the list, not the end.